Министерство образования Республики Беларусь

Учреждение образования
«Полоцкий государственный университет имени Евфросинии Полоцкой»

Факультет информационных технологий
Кафедра вычислительных систем

ОТЧЕТ
По лабораторной работе №2
на тему:
«Построение физической модели базы данных предметной области «Архив конструкторской документации»»
по дисциплине
«Базы данных»

Выполнил — Литвинов Р.А., студент группы 25-СТ

Проверил — Гордевич Д.А., Преподаватель-стажер

Полоцк, 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы – научиться переходить от ER-модели к физической модели базы данных: представить сущности отношениями (таблицами), выбрать типы данных, первичные и внешние ключи, избавиться от связей 1:1 и «многие-ко-многим», найти и обезвредить циклы и убедиться, что схема удовлетворяет 1, 2 и 3 нормальным формам.

2 ЗАДАНИЕ

Вариант 7. Тема – архив конструкторской документации. Исходные данные – итоговая ER-модель лабораторной работы №1. С помощью свободного редактора схем draw.io требуется разработать физическую модель БД: создать отношения, устранить связи 1:1 и M:N (последние – введением таблицы-посредника), определить ключи и при необходимости ввести суррогатные, нарисовать связи, устранить циклы и проверить схему на 1НФ, 2НФ и 3НФ.

Физическая модель приведена на рисунке 1, файл draw.io приложен к отчёту. Чтобы убедиться, что модель работоспособна, она создана в СУБД PostgreSQL 17: скрипт выполнен без ошибок, таблицы заполнены примером данных из лабораторной работы №1, после чего проверены попытки вставить заведомо неверные строки.

3 АЛГОРИТМ РЕШЕНИЯ

– каждая сильная сущность ER-модели (изделие, подразделение, сотрудник, извещение, роль подписи) превращена в таблицу, атрибуты – в столбцы с типами данных;

– слабые сущности «Документ», «Изменение» и «Подпись» превращены в таблицы, в которые добавлены ключи сущностей-владельцев;

– для каждой таблицы выбран первичный ключ: короткие устойчивые коды оставлены естественными ключами, для изделия, документа и извещения введены суррогатные ключи, естественные ключи закреплены ограничением UNIQUE;

– каждая связь 1:N реализована внешним ключом в таблице на стороне N;

– две связи M:N – рекурсивная «Входит в состав» и «Абонент» – заменены таблицами-посредниками product_part и subscription; связей 1:1 нет;

– найдены циклы в схеме; проверено, что они не блокируют вставку, а цикл с двусмысленностью закрыт триггером;

– схема проверена на 1НФ, 2НФ и 3НФ, создана и испытана в PostgreSQL.

4 ИСХОДНАЯ ER-МОДЕЛЬ

Итоговая ER-модель лабораторной работы №1 содержит пять сильных сущностей (изделие, подразделение, сотрудник, извещение, роль подписи) и

три слабые: документ (владелец – изделие), изменение (владелец – документ) и подпись (владельцы – документ и роль). Связи: девять связей 1:N и две связи M:N – рекурсивная «Входит в состав» с атрибутом «Количество» и «Абонент» с атрибутами «Экземпляров» и «Дата постановки на учёт». Промежуточная подпись с собственным номером в итоговую модель не вошла.

Правила перехода к таблицам: сущность – таблица, атрибут – столбец, ключ – первичный ключ, связь 1:N – внешний ключ в таблице на стороне N, атрибуты связи 1:N переходят туда же. Связь M:N превращается в отдельную таблицу, первичный ключ которой составлен из внешних ключей участников, а атрибуты связи становятся её столбцами. Связь 1:1 устраняется слиянием таблиц или внешним ключом с ограничением UNIQUE.

5 ОТНОШЕНИЯ ФИЗИЧЕСКОЙ МОДЕЛИ

Таблицы и столбцы названы латиницей в нижнем регистре с подчёркиваниями, чтобы в SQL их не нужно было заключать в кавычки. На схеме в заголовке каждой таблицы указано русское название сущности.

Таблица 1 – Отношения физической модели

Таблица	Сущность ER-модели	Первичный ключ	Внешние ключи
department	Подразделение	dept_code (естеств.)	–
employee	Сотрудник	employee_id (табельный номер)	dept_code
product	Изделие	product_id (супр.)	dept_code
product_part	Связь «Входит в состав» (M:N)	assembly_id + part_id	assembly_id, part_id
document	Документ (слабая)	document_id (супр.)	product_id
notice	Извещение	notice_id (супр.)	dept_code
doc_change	Изменение (слабая)	document_id + change_no	document_id, notice_id
subscription	Связь «Абонент» (M:N)	dept_code + document_id	dept_code, document_id
sign_role	Роль подписи	role_code (естеств.)	–
signature	Подпись (слабая)	document_id + role_code	document_id, role_code, employee_id

Типы данных подобраны по смыслу значений: код подразделения – CHAR(3), так как он всегда из трёх цифр; обозначение изделия – VARCHAR(20) с проверкой формата по шаблону «АБВГ.XXXXXX.XXX»; даты – DATE; количество, число листов и экземпляров – SMALLINT, этого диапазона с запасом хватает; суррогатные ключи – SERIAL. Вид изделия и

код документа – короткие строки, допустимые значения которых перечислены в CHECK.

Таблица 2 – Ограничения, кроме первичных и внешних ключей

Таблица	Ограничение	Тип	Назначение
product	designation	UNIQUE, CHECK	Естественный ключ изделия и формат обозначения
document	product_id, doc_code	UNIQUE	Естественный ключ слабого документа из ЛР1
notice	notice_no	UNIQUE	Номер извещения не повторяется
doc_change	document_id, notice_id	UNIQUE	Одно извещение меняет документ один раз
product	kind из списка	CHECK	Домен «вид изделия»
document	doc_code из списка, sheets > 0	CHECK	Коды по ГОСТ 2.102, контроль значений
product_part	assembly_id ≠ part_id, quantity > 0	CHECK	Изделие не входит само в себя
subscription	copies > 0	CHECK	Контроль значений

6 КЛЮЧИ

Первичный ключ – столбец или набор столбцов, значение которого однозначно определяет строку таблицы и не может быть NULL. Суррогатный ключ – искусственное целое число, не несущее смысла в предметной области; он короток, никогда не меняется и удобен как цель внешних ключей.

– Естественные ключи оставлены у подразделения (трёхзначный код), сотрудника (табельный номер) и роли подписи (код роли «РАЗР», «ПРОВ» и т.д.). Они короткие, уникальны и в архиве не меняются.

– Суррогатные ключи введены у изделия, документа и извещения. Обозначение изделия – строка до 20 символов, при переобозначении изделия она меняется, а на изделие ссылаются три внешних ключа. Ключ документа из ЛР1 – обозначение изделия и код документа, на документ ссылаются три таблицы, и каждой пришлось бы хранить пару столбцов. Номер извещения – длинная строка, на неё ссылаются изменения.

– Составные ключи оставлены у таблиц, на которые никто не ссылается: product_part (assembly_id, part_id), subscription (dept_code, document_id), doc_change (document_id, change_no) и signature (document_id,

role_code). Ключ подписи напрямую повторяет итог ЛР1 – одна подпись на документ в каждой роли.

Ввод суррогатного ключа не отменяет естественный: он закреплён ограничением UNIQUE. Без него в таблице document можно было бы завести два сборочных чертежа одного блока питания под разными document_id.

7 СВЯЗИ

Связь в физической модели реализуется внешним ключом: значение столбца дочерней таблицы обязано совпадать с первичным ключом строки родительской таблицы. На схеме у дочерней таблицы стоит N, у родительской – 1.

Таблица 3 – Связи физической модели

Связь ER-модели	Внешний ключ	Тип	Обязательность
Работает	employee.dept_code → department	1 : N	NOT NULL
Разрабатывает	product.dept_code → department	1 : N	NOT NULL
Выпускает	notice.dept_code → department	1 : N	NOT NULL
Входит в состав (сборка)	product_part.assembly_id → product	1 : N	NOT NULL, часть PK
Входит в состав (часть)	product_part.part_id → product	1 : N	NOT NULL, часть PK
Описывает	document.product_id → product	1 : N	NOT NULL
Касается	doc_change.document_id → document	1 : N	NOT NULL, часть PK
Вносит	doc_change.notice_id → notice	1 : N	NOT NULL
Абонент (подразделение)	subscription.dept_code → department	1 : N	NOT NULL, часть PK
Абонент (документ)	subscription.document_id → document	1 : N	NOT NULL, часть PK
Заверяет	signature.document_id → document	1 : N	NOT NULL, часть PK
В роли	signature.role_code → sign_role	1 : N	NOT NULL, часть PK
Ставит	signature.employee_id → employee	1 : N	NOT NULL

7.1 Устранение связей 1:1 и M:N

Связей 1:1 в ER-модели нет. Если бы, например, у каждого документа был ровно один «паспорт хранения» с номером стеллажа и ячейки, его

столбцы были бы перенесены в таблицу document либо паспорт получил бы внешний ключ document_id с ограничением UNIQUE.

Связей M:N две. Рекурсивная связь «Входит в состав» заменена таблицей product_part: в ней два внешних ключа на одну и ту же таблицу product – на сборку и на её часть – и атрибут quantity. Связь «Абонент» заменена таблицей subscription с внешними ключами на подразделение и документ и атрибутами copies и since_date. Каждая связь M:N распалась на две связи 1:N, а повторить одну и ту же пару дважды не позволяет составной первичный ключ.

7.2 Циклы

Цикл – замкнутый путь по внешним ключам. Опасен цикл из обязательных ссылок, в котором ни одну таблицу нельзя заполнить первой, и цикл с двусмысленностью, когда до одной и той же строки можно дойти двумя путями и пути могут привести к разным значениям.

Циклов первого вида нет: все ссылки направлены к справочникам, и таблицы заполняются по порядку – подразделения и роли, сотрудники, изделия и их состав, документы, извещения, изменения, подписки и подписи. Рекурсивная таблица product_part тоже не мешает вставке: сначала создаются изделия, потом связи между ними. На уровне данных в составе изделий мог бы возникнуть цикл «A входит в B, B входит в A»; его запрещает триггер check_part, который обходит дерево состава рекурсивным запросом.

Цикл второго вида показан на рисунке 2: doc_change → notice → department и doc_change → document → product → department. Первый путь даёт подразделение, выпустившее извещение, второй – разработчика изделия. По правилу из раздела 9 ЛР1 они должны совпадать. Удалить одну из связей нельзя – каждая хранит свой факт, поэтому совпадение проверяет триггер check_change. Он же контролирует порядок номеров изменений и то, что изменение не внесено раньше даты извещения.

Аналогичные пути есть через сотрудника подписи и через подписку: signature → employee → department и subscription → department. Здесь различие путей допустимо – нормоконтролёр может работать в другом отделе, а абонентом обычно является цех, а не разработчик, поэтому ограничений не требуется. Правило «разработчик не проверяет свой документ» закрыто триггером check_signature.

8 НОРМАЛИЗАЦИЯ

– 1НФ: все значения атомарны, повторяющихся групп нет. Состав изделия хранится строками product_part, а не списком частей в одном поле; подписи – строками signature, а не пятью столбцами «разработал, проверил…» в таблице document. Перечень изменённых листов хранится одной строкой, потому что по отдельным листам он не обрабатывается.

– 2НФ: составные ключи у product_part, subscription, doc_change и signature. Их неключевые столбцы зависят от всего ключа: количество – от пары «сборка + часть», число копий – от пары «подразделение + документ», извещение и дата – от пары «документ + номер изменения», подписант и дата – от пары «документ + роль». Название документа или подразделения в эти таблицы не копируется.

– 3НФ: транзитивных зависимостей нет. Разработчик хранится у изделия, а не у документа; подразделение – у сотрудника, а не у подписи; дата и причина изменения – у извещения, а не в каждой строке doc_change; наименование роли – в sign_role.

Дата подписи и дата внесения изменения не являются дублированием: это факты конкретного события, которые нельзя вывести из других таблиц.

9 ФИЗИЧЕСКАЯ МОДЕЛЬ

Обозначения на схеме: в заголовке таблицы – русское название сущности и имя таблицы. Голубой заголовок – таблица из сильной сущности, оранжевый – из слабой, зелёный – таблица-посредник, полученная из связи M:N. PK – первичный ключ (выделен фоном и подчёркнут), FK – внешний ключ, U – уникальность (U1 – столбцы одного составного ограничения). Справа указан тип данных. Линия соединяет внешний ключ с таблицей, на которую он ссылается.

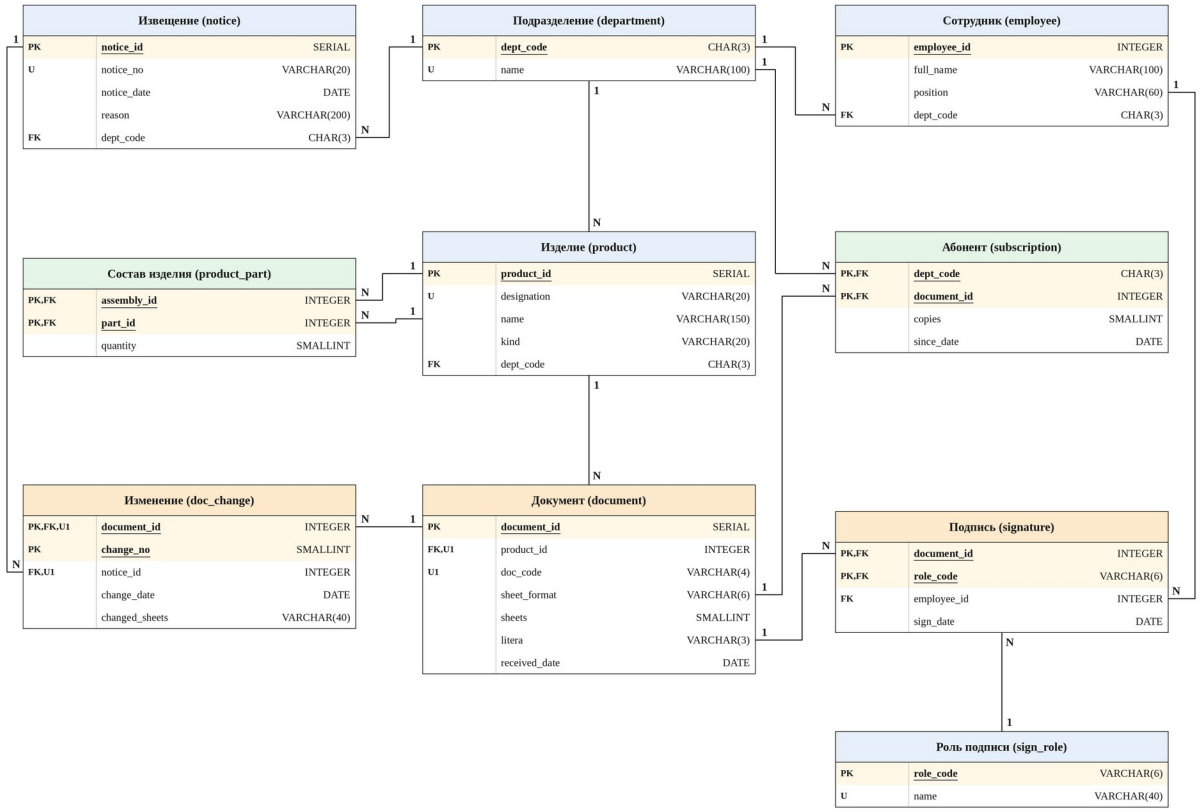


Рисунок 1 – Физическая модель базы данных «Архив конструкторской документации»

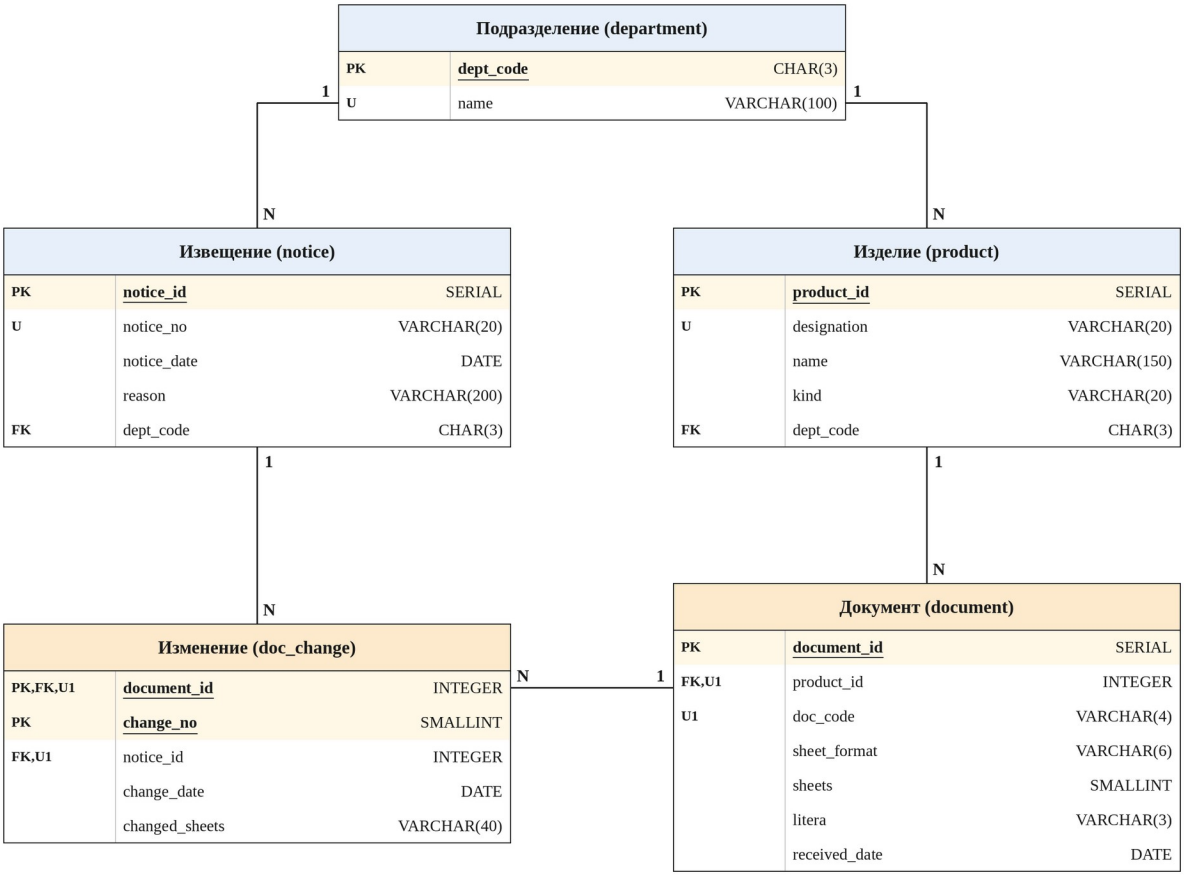


Рисунок 2 – Фрагмент с циклом: подразделение извещения и разработчик изделия

10 ПРОВЕРКА В СУБД

Скрипт из приложения А выполнен в PostgreSQL 17 без ошибок, после чего таблицы заполнены примером из таблицы 4 лабораторной работы №1 и выполнены ошибочные вставки. Все они отклонены СУБД.

Таблица 4 – Проверка ограничений

№	Попытка	Что сработало	Результат
1	Корпус в составе платы в количестве 0	CHECK quantity > 0	Отклонено
2	Блок питания в составе своего корпуса	Триггер check_part	Отклонено: цикл
3	Второй сборочный чертёж БП-12	UNIQUE документа	Отклонено: дубль ключа
4	Изменение схемы по извещению КБ-3	Триггер check_change	Отклонено: чужое КБ
5	Изменение 3 схемы без изменения 2	Триггер check_change	Отклонено: порядок
6	Разработчик схемы в роли «Проверил»	Триггер check_signature	Отклонено
7	Подписка подразделения 999	FOREIGN KEY dept_code	Отклонено

Корректные строки примера – состав блока питания, пять документов, изменения № 1 схемы и спецификации по извещению НПГУ.012-26, две подписки цеха № 3 и четыре подписи сборочного чертежа – вставлены без ошибок. Скрипт создания базы приведён в приложении А.

11 ВЫВОД

По ER-модели «Архив конструкторской документации» построена физическая модель из десяти таблиц. Для изделия, документа и извещения введены суррогатные ключи с сохранением естественных ключей в UNIQUE, у справочников оставлены естественные коды. Все связи реализованы внешними ключами 1:N, связей 1:1 нет, две связи M:N, включая рекурсивный состав изделия, разложены через таблицы-посредники product_part и subscription. Циклов, мешающих вставке, нет; цикл через подразделение и данные с замкнутым составом изделия контролируются триггерами. Схема находится в 3НФ и проверена в PostgreSQL.

Контрольные вопросы к лабораторной работе №2

1 Опишите алгоритм построения физической модели.

Каждая сущность ER-модели становится таблицей, а атрибуты – столбцами с конкретными типами. Для таблиц выбираются первичные ключи, длинные или изменчивые естественные ключи заменяются суррогатными. Связь 1:N превращается во внешний ключ на стороне N, связь M:N – в таблицу-посредник, связь 1:1 – в слияние таблиц или уникальный внешний ключ. После этого схема проверяется на циклы и нормальные формы.

2 Какие сущности Вы превратили в таблицы БД?

Все восемь сущностей итоговой ER-модели: подразделение, сотрудник, изделие, документ, извещение, изменение, роль подписи и подпись. Кроме того, таблицами стали две связи M:N – состав изделия (product_part) и абонентский учёт (subscription).

3 Какие первичные ключи Вы определили?

Естественные: dept_code у подразделения, табельный номер employee_id у сотрудника, role_code у роли подписи. Суррогатные: product_id, document_id, notice_id. Составные: (assembly_id, part_id), (dept_code, document_id), (document_id, change_no) и (document_id, role_code). Естественные ключи изделия, документа и извещения закреплены ограничениями UNIQUE.

4 Какие связи были реализованы и как Вы устранили наличие связей типа 1x1 MxN?

Реализовано тринадцать обязательных связей 1:N внешними ключами. Связей 1:1 в модели нет. Рекурсивная связь M:N «Входит в состав» заменена

таблицей product_part с двумя внешними ключами на product, связь M:N «Абонент» – таблицей subscription с внешними ключами на подразделение и документ.

5 Нормализована ли полученная Вами схема БД?

Да, схема в 3НФ. Все значения атомарны, списки частей и подписей вынесены в отдельные строки (1НФ). В таблицах с составным ключом неключевые столбцы зависят от всего ключа (2НФ). Неключевые столбцы не зависят друг от друга: разработчик хранится у изделия, подразделение – у сотрудника, дата и причина изменения – у извещения (3НФ).

ПРИЛОЖЕНИЕ А

Скрипт создания базы данных (PostgreSQL)

```
-- Физическая модель БД «Архив конструкторской документации»
-- (ЛР2, вариант 7), PostgreSQL
CREATE TABLE department (
    dept_code CHAR(3)      PRIMARY KEY CHECK (dept_code ~ '^[0-9]{3}$'),
    name      VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE employee (
    employee_id INTEGER      PRIMARY KEY,          -- табельный номер
    full_name   VARCHAR(100) NOT NULL,
    position    VARCHAR(60)  NOT NULL,
    dept_code   CHAR(3)      NOT NULL REFERENCES department
);
CREATE TABLE product (
    product_id  SERIAL PRIMARY KEY,
    designation VARCHAR(20)  NOT NULL UNIQUE
                CHECK (designation ~ '^[А-Я]{4}\.[0-9]{6}\.[0-9]{3}$'),
    name        VARCHAR(150) NOT NULL,
    kind        VARCHAR(20)  NOT NULL
                CHECK (kind IN ('деталь', 'сборочная единица',
                                'комплекс', 'комплект')),
    dept_code   CHAR(3)      NOT NULL REFERENCES department -- разработчик
);
CREATE TABLE product_part (
    assembly_id INTEGER  NOT NULL REFERENCES product,
    part_id     INTEGER  NOT NULL REFERENCES product,
    quantity    SMALLINT NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (assembly_id, part_id),
    CHECK (assembly_id <> part_id)
);
CREATE TABLE document (
    document_id   SERIAL PRIMARY KEY,
    product_id    INTEGER     NOT NULL REFERENCES product,
    doc_code      VARCHAR(4)  NOT NULL
                  CHECK (doc_code IN ('ЧД', 'СП', 'СБ', 'ВО',
                                      'ЭЗ', 'ПЭ3', 'ТУ', 'ПЗ')),
    sheet_format  VARCHAR(6)  NOT NULL,
    sheets        SMALLINT    NOT NULL CHECK (sheets > 0),
    litera        VARCHAR(3)  NOT NULL,
    received_date DATE        NOT NULL,
    UNIQUE (product_id, doc_code)
);
CREATE TABLE notice (
    notice_id   SERIAL PRIMARY KEY,
    notice_no   VARCHAR(20)  NOT NULL UNIQUE,
    notice_date DATE         NOT NULL,
    reason      VARCHAR(200) NOT NULL,
    dept_code   CHAR(3)      NOT NULL REFERENCES department
);
CREATE TABLE doc_change (
    document_id    INTEGER     NOT NULL REFERENCES document,
    change_no      SMALLINT    NOT NULL CHECK (change_no > 0),
    notice_id      INTEGER     NOT NULL REFERENCES notice,
    change_date    DATE        NOT NULL,
    changed_sheets VARCHAR(40) NOT NULL,
    PRIMARY KEY (document_id, change_no),
    UNIQUE (document_id, notice_id)
);
CREATE TABLE subscription (
    dept_code   CHAR(3)  NOT NULL REFERENCES department,
```

```
    document_id INTEGER  NOT NULL REFERENCES document,
    copies      SMALLINT NOT NULL CHECK (copies > 0),
    since_date  DATE     NOT NULL,
    PRIMARY KEY (dept_code, document_id)
);
CREATE TABLE sign_role (
    role_code VARCHAR(6)  PRIMARY KEY,
    name      VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE signature (
    document_id INTEGER    NOT NULL REFERENCES document,
    role_code   VARCHAR(6) NOT NULL REFERENCES sign_role,
    employee_id INTEGER    NOT NULL REFERENCES employee,
    sign_date   DATE       NOT NULL,
    PRIMARY KEY (document_id, role_code)
);
CREATE INDEX ix_product_dept   ON product(dept_code);
CREATE INDEX ix_part_part      ON product_part(part_id);
CREATE INDEX ix_change_notice  ON doc_change(notice_id);
CREATE INDEX ix_signature_emp  ON signature(employee_id);
CREATE INDEX ix_subscr_doc     ON subscription(document_id);

-- Правила, которые не выражаются внешними ключами (замыкание циклов)
CREATE FUNCTION check_part() RETURNS trigger AS $$
BEGIN
    IF EXISTS (WITH RECURSIVE up(id) AS (
                    SELECT NEW.assembly_id
                    UNION
                    SELECT pp.assembly_id FROM product_part pp
                      JOIN up ON pp.part_id = up.id)
                SELECT 1 FROM up WHERE id = NEW.part_id) THEN
        RAISE EXCEPTION 'Цикл в составе: изделие входит само в себя';
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_part BEFORE INSERT OR UPDATE ON product_part
    FOR EACH ROW EXECUTE FUNCTION check_part();

CREATE FUNCTION check_change() RETURNS trigger AS $$
DECLARE n notice; p product;
BEGIN
    SELECT * INTO n FROM notice WHERE notice_id = NEW.notice_id;
    SELECT pr.* INTO p FROM product pr
      JOIN document d ON d.product_id = pr.product_id
     WHERE d.document_id = NEW.document_id;
    IF n.dept_code <> p.dept_code THEN
        RAISE EXCEPTION 'Извещение выпущено не разработчиком изделия';
    END IF;
    IF NEW.change_date < n.notice_date THEN
        RAISE EXCEPTION 'Изменение внесено раньше даты извещения';
    END IF;
    IF NEW.change_no <> 1 + COALESCE((SELECT MAX(change_no)
            FROM doc_change WHERE document_id = NEW.document_id
             AND change_no <> NEW.change_no), 0) THEN
        RAISE EXCEPTION 'Номер изменения должен быть следующим';
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_change BEFORE INSERT OR UPDATE ON doc_change
    FOR EACH ROW EXECUTE FUNCTION check_change();

CREATE FUNCTION check_signature() RETURNS trigger AS $$
```

```
BEGIN
    IF NEW.role_code IN ('РАЗР', 'ПРОВ') AND EXISTS (
            SELECT 1 FROM signature
             WHERE document_id = NEW.document_id
               AND employee_id = NEW.employee_id
               AND role_code IN ('РАЗР', 'ПРОВ')
               AND role_code <> NEW.role_code) THEN
        RAISE EXCEPTION 'Разработчик не может проверять свой документ';
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_signature BEFORE INSERT OR UPDATE ON signature
    FOR EACH ROW EXECUTE FUNCTION check_signature();
```